

# Elevate Labs Internship Deliverable

SQL Developer Internship • Task 5 Submission

|               |                   |             |                                  |
|---------------|-------------------|-------------|----------------------------------|
| Submitted By: | G. Pavana Deepika | Task Name:  | Task 5: SQL Joins                |
| Domain:       | SQL Developer     | Tools Used: | MySQL / SQLite / MySQL Workbench |

## 1. Table Schema Creation & Data Setup

```
-- Create Customers Table
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(50),
  City VARCHAR(50)
);

-- Create Orders Table
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  OrderDate DATE,
  Amount DECIMAL(10, 2)
);

-- Insert Sample Records
INSERT INTO Customers (CustomerID, CustomerName, City) VALUES
(1, 'Alice Smith', 'New York'),
(2, 'Bob Jones', 'Los Angeles'),
(3, 'Charlie Brown', 'Chicago'),
(4, 'Diana Prince', 'Miami');

INSERT INTO Orders (OrderID, CustomerID, OrderDate, Amount) VALUES
(101, 1, '2026-01-15', 250.00),
(102, 2, '2026-01-18', 150.50),
(103, 1, '2026-01-20', 300.00),
(104, 5, '2026-01-22', 450.00);
```

## 2. SQL Join Queries

### A. INNER JOIN

```
-- Returns records with matching values in both tables
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID, Orders.Amount
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

### B. LEFT JOIN (LEFT OUTER JOIN)

```
-- Returns all records from left table (Customers) and matched records from right table (Orders)
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID, Orders.Amount
```

```
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

### C. RIGHT JOIN (RIGHT OUTER JOIN)

```
-- Returns all records from right table (Orders) and matched records from left table (Customers)
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID, Orders.Amount
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

### D. FULL JOIN (FULL OUTER JOIN)

```
-- Full Join implementation using UNION for compatibility with SQLite/MySQL
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID, Orders.Amount
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
UNION
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID, Orders.Amount
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

## 3. Interview Questions & Technical Answers

#### 1. What is the main difference between INNER and LEFT JOIN?

**INNER JOIN** returns only rows with matching key values in both tables. **LEFT JOIN** returns all rows from the left table regardless of a match, filling unmapped right table columns with NULL values.

#### 2. What is a FULL OUTER JOIN?

A **FULL OUTER JOIN** combines the result set of both **LEFT** and **RIGHT** joins. It retains all records from both tables, populating missing values with NULL on whichever side lacks a matching row.

#### 3. Can joins be nested?

Yes, joins can be nested using parentheses or subqueries/derived tables in the **FROM**/**JOIN** clauses to aggregate or restrict data prior to joining.

#### 4. How do you join more than 2 tables?

You can sequence multiple **JOIN** clauses consecutively (e.g., **FROM TableA JOIN TableB ON ... JOIN TableC ON ...**).

#### 5. What is a CROSS JOIN?

A **CROSS JOIN** generates a Cartesian product of the involved tables, combining each row of the first table with every row of the second table without requiring an explicit join condition.

#### 6. What is a NATURAL JOIN?

A **NATURAL JOIN** implicitly joins tables based on all columns having matching names and compatible data types, omitting the explicit **ON** clause.

### **7. Can you join tables without a foreign key?**

Yes, foreign key constraints are not strictly necessary to execute joins; logical relational equivalences on columns with shared values/types are sufficient.

### **8. What is a self-join?**

A self-join occurs when a single table is joined to itself using table aliases, useful for evaluating hierarchical data within the same entity.

### **9. What causes a Cartesian product?**

A Cartesian product is produced when joining tables without defining an explicit join predicate (missing ON or WHERE clause) or by explicitly invoking a CROSS JOIN.

### **10. How to optimize join performance?**

Optimize joins by adding indexes to join/foreign key columns, pre-filtering datasets using subqueries, avoiding SELECT \*, and ensuring joined data types match identically.